

LAPORAN PRAKTIKUM 4
PEMOGRAMAN BERORIENTASI OBJEK

Disusun Oleh:

Dinda Amelia

(2511531020)

Dosen Pengampu:

Nurfiah, S.ST, M.Kom.

Asisten Praktikum:

Ezza Addini



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG 2026

KATA PENGANTAR

Segala puji bagi Allah SWT yang telah memberikan kesempatan serta kemudahan kepada penulis untuk dapat menyelesaikan Laporan Praktikum pada mata kuliah Struktur Data, sehingga Laporan Praktikum ini dapat dikumpulkan dengan tepat waktu. Atas rahmat dan karunianya Laporan Praktikum dapat terselesaikan dengan baik. Sholawat serta salam juga penulis sampaikan kepada baginda tercinta yaitu Nabi Muhammad SAW, yang kita nantikan syafa'atnya di akhirat nanti. Laporan Praktikum ini bertujuan untuk menambah wawasan para pembaca untuk lebih memperdalam ilmu yang ada pada makalah ini.

Laporan pratikum ini disusun sebagai bentuk penanggung jawaban atas pelaksanaan kegiatan pratikum mata kuliah Pemograman Berorientasi Objek yang membahas tentang Ekspansi Produk Keuangan (Inheritance).

Dalam penyusunan Laporan Praktikum ini, penulis mengalami banyak kesulitan dan penulis menyadari bahwa Laporan Praktikum ini masih jauh dari kesempurnaan. Untuk itu, penulis sangat mengharapkan kritik dan saran yang membangun demi kesempurnaan pada Laporan Praktikum ini.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I	1
PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan.....	2
1.3 Manfaat.....	2
1.4 Link repository Github	2
BAB II.....	3
PEMBAHASAN	3
2.1 Dasar Teori	3
2.2Langkah kerja	4
2.2.1 Class 1 “Rekening”	4
2.2.2 Class 2 “RekeningGiro”.....	7
2.2.3 Class 3“RekeningTabungan”	8
2.2.4Class 4 “Rekening VIP”	9
2.2.5 Class 5 “Transaksi”	11
2.2.6 Class 6 “Main”	12
2.3 Latihan.....	15
2.3.1 TUGAS 1	15
2.3.2 TUGAS 2	16
2.4 OUTPUT	17
BAB III	19
KESIMPULAN	19
DAFTAR PUSTAKA	20

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengembangan sistem layanan perbankan modern menuntut ketersediaan variasi produk keuangan yang beragam, seperti Rekening Tabungan dengan bunga bulanan, Rekening Giro dengan fasilitas *overdraft*, maupun Rekening VIP. Pembuatan kelas-kelas produk tersebut secara terpisah dari awal berpotensi menyebabkan redundansi kode yang melanggar prinsip *Don't Repeat Yourself* (DRY), mengingat logika dasar seperti pembuatan akun, verifikasi PIN, serta pencatatan mutasi transaksi bersifat identik. Oleh karena itu, konsep *Inheritance* (pewarisan) dalam Pemrograman Berorientasi Objek hadir sebagai solusi untuk mewujudkan *code reusability* dengan menetapkan kelas induk Rekening dan menurunkan atribut serta metodenya ke kelas-kelas anak yang lebih spesifik.

Melalui mekanisme pewarisan ini, penggunaan kata kunci *extends* dan *super* memungkinkan penambahan fitur khusus pada *subclass* tanpa harus menulis ulang kode dasar dari *superclass*. Penerapan modifier *protected* juga menjaga enkapsulasi data sensitif sembari memberikan akses variabel secara aman kepada kelas turunan. Ditambah dengan teknik *upcasting* dan *downcasting*, berbagai objek rekening yang berbeda dapat dikelola secara fleksibel dalam satu koleksi terpadu. Praktikum Modul 4 ini dilaksanakan untuk memberikan pemahaman mendalam mengenai implementasi *Inheritance*, pengorganisasian kelas turunan, serta manipulasi tipe referensi pada Java melalui studi kasus sistem ekspansi produk perbankan.

1.2 Tujuan

1. Memahami prinsip Code Reusability melalui Pewarisan (Inheritance).
2. Membedakan Superclass dan Subclass.
3. Menggunakan keyword extends untuk mewariskan sifat, serta super() untuk memanggil konstruktor induk.
4. Menerapkan Upcasting dan Downcasting pada koleksi objek.

1.3 Manfaat

1. Meningkatkan efisiensi pengembangan perangkat lunak karena kode yang sudah ada pada kelas induk dapat digunakan kembali oleh kelas anak tanpa perlu ditulis dari awal.
2. Memudahkan perancangan struktur program secara hirarkis antara kelas umum (*Superclass*) dan kelas spesifik (*Subclass*) agar pengelolaan komponen sistem menjadi lebih terorganisir.
3. Memungkinkan kelas turunan mengambil atribut serta metode dari kelas induk sekaligus melakukan inisialisasi data secara konsisten dan aman.
4. Memungkinkan berbagai jenis objek turunan dikelola secara fleksibel dalam satu koleksi seragam serta diakses kembali ke tipe spesifiknya saat dibutuhkan.

1.4 Link repository Github

1. https://github.com/faridaarifin91-sudo/PrakPBO_B_2511531020/tree/Praktikum4/PrakPBO/src/pekan4

BAB II

PEMBAHASAN

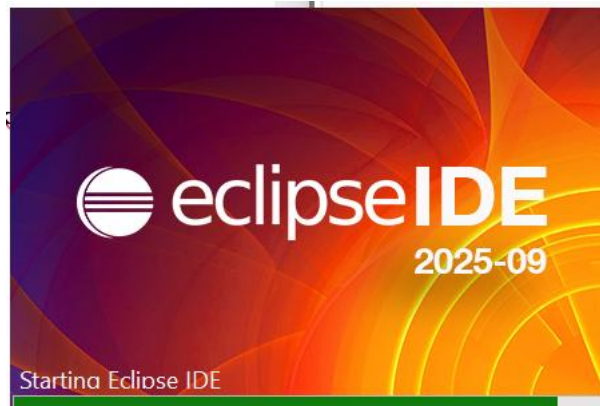
2.1 Dasar Teori

Pewarisan (*Inheritance*) merupakan pilar Pemrograman Berorientasi Objek yang memungkinkan pembuatan kelas anak (*subclass*) berdasarkan kelas induk (*superclass*) guna menerapkan prinsip *code reusability* dan menghindari redundansi kode (DRY). Melalui relasi "*Is-A*", *subclass* mewarisi atribut serta metode umum milik *superclass*, sembari mempertahankan fleksibilitas untuk menambahkan fitur spesifiknya sendiri. Penerapan modifier *protected* juga berperan penting untuk memberi hak akses variabel secara aman kepada kelas turunan tanpa membuka akses tersebut ke publik.

Proses pewarisan diimplementasikan menggunakan kata kunci *extends*, sementara kata kunci *super()* digunakan di baris pertama konstruktor anak untuk menginisialisasi kelas induk. Selain itu, relasi pewarisan memungkinkan konversi tipe referensi berupa *Upcasting* (menyimpan objek anak ke variabel induk secara otomatis) dan *Downcasting* (mengubah kembali tipe referensi ke kelas anak secara eksplisit). Teknik *Upcasting* memudahkan penyimpanan berbagai varian objek dalam satu koleksi seragam seperti *ArrayList*, sedangkan *Downcasting* yang didahului pengecekan operator *instanceof* memastikan pemanggilan metode spesifik kelas anak berjalan dengan aman.

2.2 Langkah kerja

1. jalankan aplikasi Eclipse IDE pada komputer/laptop.



2. Buka project PrakPBO_B_2511531020 dan pastikan berada dalam package Pekan4.

2.2.1 Class 1 “Rekening”

1. Kode Program

```
PrakPBO_B_2511531020 ▸ src ▸ pekan4 ▸ Rekening ▸ Rekening(String, String, double, String)
1 package pekan4;
2
3 import java.util.ArrayList;
4
5 public class Rekening {
6     // 1. Atribut private tetap untuk data yang sangat sensitif
7     private String nomorRekening;
8     private String namaPemilik;
9     private String pin;
10
11     // 2. Mengubah modifier menjadi protected agar dapat diakses oleh Subclass
12     protected double saldo;
13     protected ArrayList<Transaksi> riwayatTransaksi;
14
15     // Constructor Superclass
16     public Rekening(String nomor, String nama, double saldoAwal, String pinAwal) {
17         this.nomorRekening = nomor;
18         this.namaPemilik = nama;
19         this.saldo = saldoAwal;
20
21         if (pinAwal.length() == 6) {
22             this.pin = pinAwal;
23         } else {
24             System.out.println("Peringatan: PIN harus 6 digit! Menggunakan PIN default 123456");
25             this.pin = "123456";
26         }
27
28         this.riwayatTransaksi = new ArrayList<>();
29         System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat.");
30     }
31
32     public String getNomorRekening() { return nomorRekening; }
33     public String getNamaPemilik() { return namaPemilik; }
34     public double getSaldo() { return saldo; }
```

```

36 public boolean otentikasi(String inputPin) {
37     return this.pin.equals(inputPin);
38 }
39
40 public void setorTunai(double nominal) {
41     if (nominal > 0) {
42         saldo += nominal;
43         riwayatTransaksi.add(new Transaksi("TRX-S-" + System.currentTimeMillis(), "Kredit", nominal));
44         System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
45     } else {
46         System.out.println("Gagal: nominal setor harus lebih dari 0!");
47     }
48 }
49
50 public void tarikTunai(double nominal) {
51     if (nominal < 10000) {
52         System.out.println("Transaksi Gagal: Minimal nominal penarikan 10.000");
53     } else if (nominal > saldo) {
54         System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
55     } else {
56         saldo -= nominal;
57         riwayatTransaksi.add(new Transaksi("TRX-T-" + System.currentTimeMillis(), "Debit", nominal));
58         System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
59     }
60 }
61
62 public void cekInformasi() {
63     System.out.println("--- INFO REKENING ---");
64     System.out.println("No. Rekening : " + nomorRekening);
65     System.out.println("Nama Pemilik : " + namaPemilik);
66     System.out.println("Saldo Akhir : Rp" + saldo);
67     System.out.println("-----");
68 }
69
70 public void cetakMutasi() {
71     System.out.println("\n===== MUTASI REKENING =====");
72     System.out.println("No. Rekening : " + nomorRekening);
73     System.out.println("Nama Pemilik : " + namaPemilik);
74     System.out.println("-----");
75     if (riwayatTransaksi.isEmpty()) {
76         System.out.println("Belum ada riwayat transaksi.");
77     } else {
78         for (Transaksi t : riwayatTransaksi) {
79             t.cetakDetail();
80         }
81     }
82     System.out.println("-----");
83 }
84 }

```

2. Analisis Langkah

a. Struktur Program

- Berfungsi sebagai *superclass* (kelas induk) yang mendefinisikan atribut utama, enkapsulasi data sensitif, serta fungsi-fungsi perbankan dasar (pembuatan akun, validasi PIN, setor tunai, tarik tunai, cek informasi, dan cetak mutasi).

b. Elemen Sintaks Utama

- `private String nomorRekening, namaPemilik, pin` → atribut yang dikunci dengan akses `private` untuk menjaga kerahasiaan data sensitif agar tidak bisa diubah langsung dari luar kelas.
- `protected double saldo` dan `protected ArrayList<Transaksi> riwayatTransaksi` → atribut yang diberi modifier `protected` agar *subclass* (seperti *RekeningTabungan* atau *RekeningGiro*) dapat mengakses dan memanipulasi data saldo serta riwayat transaksi secara langsung.

- `public Rekening(...)` →konstruktor untuk menginisialisasi nilai awal akun, serta melakukan validasi panjang PIN (wajib 6 digit, jika tidak maka diset ke default "123456").
- `otentikasi(String inputPin)` →metode verifikasi kebenaran PIN yang dimasukkan pengguna.
- `setorTunai(double nominal)` dan `tarikTunai(double nominal)` →metode transaksi yang memvalidasi nominal, memperbarui nilai saldo, serta menambahkan catatan objek Transaksi baru ke `riwayatTransaksi`.
- `cekInformasi()` dan `cetakMutasi()` →metode untuk menampilkan rincian akun serta mencetak seluruh riwayat transaksi yang tersimpan.

c. Alur Kerja Sintaks

- Saat objek `Rekening` diinstansiasi melalui konstruktor, sistem memvalidasi PIN; jika valid (6 digit), PIN disimpan, lalu daftar `riwayatTransaksi` diinisialisasi sebagai `ArrayList` kosong.
- Ketika `setorTunai()` atau `tarikTunai()` dipanggil, program terlebih dahulu mengecek aturan nominal (misal: minimal tarik 10.000 dan saldo mencukupi). Jika memenuhi syarat, nilai variabel saldo diperbarui dan objek Transaksi baru otomatis ditambahkan ke `riwayatTransaksi`.
- Metode `cetakMutasi()` akan melakukan perulangan (*looping*) pada `riwayatTransaksi` dan memanggil metode `cetakDetail()` dari tiap-tiap objek transaksi untuk menampilkan riwayat ke konsol.

2.2.2 Class 2 “RekeningGiro”

1. Kode Program

```
IterasiQueu... RekeningTabu... Rekening.java RekeningGiro... X Main.java Transaksi.java RekeningVIP... »
PrakPBO_B_2511531020 ▶ src ▶ pekan4 ▶ RekeningGiro ▶ getBatasOverdraft() : double
1 package pekan4;
2
3 public class RekeningGiro extends Rekening {
4     private double batasOverdraft;
5
6     public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal, double batasOverdraft) {
7         //Memanggil inisialisasi dasar dari Superclass
8         super(nomor, nama, saldoAwal, pinAwal);
9         this.batasOverdraft = batasOverdraft;
10    }
11
12    //Getter khusus Giro
13    public double getBatasOverdraft() {
14        return batasOverdraft;
15    }
16 }
17
```

2. Analisis Langkah

a. Struktur Program

- Berfungsi sebagai *subclass* (kelas anak) turunan dari Rekening yang merepresentasikan produk Giro Bisnis dengan penambahan atribut khusus berupa limit pinjaman/cerukan (*batasOverdraft*).

b. Elemen Sintaks Utama

- `extends Rekening` → kata kunci untuk mewarisi seluruh atribut, metode, serta perilaku dari *superclass* Rekening.
- `private double batasOverdraft` → atribut khusus dengan enkapsulasi `private` untuk menyimpan limit batas penarikan melebihi saldo (*overdraft*).
- `super(nomor, nama, saldoAwal, pinAwal)` → instruksi di baris pertama konstruktor untuk memanggil konstruktor induk Rekening guna menginisialisasi nomor rekening, nama pemilik, saldo awal, dan PIN.
- `public double getBatasOverdraft()` → metode *getter* untuk mengakses nilai atribut `batasOverdraft` dari luar kelas.

c. Alur Kerja Sintaks

- Saat objek RekeningGiro diinstansiasi, program mengeksekusi instruksi `super(...)` terlebih dahulu untuk menjalankan validasi PIN dan inisialisasi atribut dasar pada *superclass* Rekening.
- Nilai parameter `batasOverdraft` kemudian disimpan ke dalam atribut `this.batasOverdraft` milik objek RekeningGiro.

- Objek ini dapat menggunakan seluruh fungsi transaksi umum milik Rekening (seperti setorTunai, cekInformasi, dll.) sekaligus menyediakan akses ke informasi limit *overdraft* via metode `getBatasOverdraft()`.

2.2.3 Class 3“RekeningTabungan”

1. Kode Program

```

1 package pekan4;
2
3 public class RekeningTabungan extends Rekening {
4     // Atribut spesifik produk tabungan
5     private double sukuBunga;
6
7     // Constructor Subclass
8     public RekeningTabungan(String nomor, String nama, double saldoAwal, String pinAwal, double sukuBunga) {
9         // super() memanggil konstruktor superclass (WAJIB pada baris pertama)
10        super(nomor, nama, saldoAwal, pinAwal);
11        this.sukuBunga = sukuBunga;
12    }
13
14
15    public void tambahBungaAkhirBulan() {
16        //menghitung bunga
17        //menghitung bisa mengakses saldo secara langsung dari class rekening
18        double nominalBunga = saldo * (sukuBunga / 100);
19        saldo += nominalBunga; // Mengakses atribut protected 'saldo' secara langsung
20
21        // Mencatat transaksi
22        String idTrx = "TRX-B-" + System.currentTimeMillis();
23        riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));
24
25        System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp" + nominalBunga);
26        System.out.println("Saldo saat ini: Rp" + saldo);
27    }
28 }

```

2. Analisis Langkah

a. Struktur Program

- Berfungsi sebagai *subclass* (kelas anak) turunan dari Rekening yang memperluas fungsi rekening biasa dengan fitur khusus berupa kalkulasi dan penambahan bunga bulanan.

b. Elemen Sintaks Utama

- `extends Rekening` → kata kunci untuk mewarisi seluruh atribut, metode, dan sifat dari *superclass* Rekening.
- `private double sukuBunga` → atribut spesifik berakses private untuk menyimpan besaran persentase bunga bulanan.
- `super(nomor, nama, saldoAwal, pinAwal)` → instruksi di baris pertama konstruktor untuk memanggil konstruktor *superclass* Rekening guna menginisialisasi atribut dasar.

- saldo dan riwayatTransaksi → pemanfaatan hak akses protected dari kelas induk yang memungkinkan subclass memanipulasi variabel tersebut secara langsung.
- tambahBungaAkhirBulan() → metode khusus untuk mengkalkulasi nominal bunga, memperbarui nilai saldo, mencatat objek Transaksi bunga, serta menampilkan konfirmasi perubahan saldo ke konsol.

c. Alur Kerja Sintaks

- Saat objek RekeningTabungan diinstansiasi, konstruktor mengeksekusi super(...) terlebih dahulu di baris pertama untuk menginisialisasi atribut induk, lalu menyimpan persentase bunga ke atribut sukuBunga.
- Ketika metode tambahBungaAkhirBulan() dipanggil, program menghitung nominalBunga dengan rumus $\text{saldo} * (\text{sukuBunga} / 100)$.
- Nilai nominalBunga tersebut secara langsung ditambahkan ke atribut saldo (berkat hak akses protected).
- Selanjutnya, sistem membuat idTrx unik berbasis timestamp (System.currentTimeMillis()), memasukkan transaksi bunga baru ke dalam daftar riwayatTransaksi, dan mencetak rincian bunga serta total saldo terbaru ke layar konsol.

2.2.4 Class 4 “Rekening VIP”

1. Kode Program (Challenge)

```

PrakPBO_B_2511531020 ▶ src ▶ pekan4 ▶ RekeningVIP ▶ RekeningVIP(String, String, double, String)
1 package pekan4;
2
3 public class RekeningVIP extends Rekening {
4     // Bonus otomatis untuk nasabah VIP
5     private static final double BONUS_VIP = 100000.0;
6
7     // Constructor Subclass RekeningVIP
8     public RekeningVIP(String nomor, String nama, double saldoAwal, String pinAwal) {
9         // Memanggil konstruktor superclass dengan menambahkan bonus ke saldo awal
10        super(nomor, nama, saldoAwal + BONUS_VIP, pinAwal);
11
12        // Mencatat transaksi bonus otomatis ke riwayatTransaksi (protected)
13        String idTrx = "TRX-BONUS-" + System.currentTimeMillis();
14        riwayatTransaksi.add(new Transaksi(idTrx, "Bonus VIP", BONUS_VIP));
15
16        System.out.println("Selamat! Anda mendapatkan bonus pembukaan akun VIP sebesar Rp" + BONUS_VIP);
17        System.out.println("Total saldo awal + bonus: Rp" + saldo);
18    }
19 }
20

```

2. Analisis Langkah

a. Struktur Program

- Berfungsi sebagai *subclass* (kelas anak) turunan dari Rekening untuk jenis produk VIP yang secara otomatis memberikan bonus saldo tambahan saat pertama kali diinstansiasi/dibuka.
- b. Elemen Sintaks Utama
- `extends Rekening` → kata kunci untuk mewarisi seluruh atribut, metode, dan perilaku dari *superclass* Rekening.
 - `private static final double BONUS_VIP = 100000.0` → konstanta berakses `private` untuk menyimpan nominal bonus tetap sebesar Rp100.000.
 - `super(nomor, nama, saldoAwal + BONUS_VIP, pinAwal)` → pemanggilan konstruktor *superclass* Rekening pada baris pertama yang langsung menambahkan nilai `saldoAwal` dengan `BONUS_VIP`.
 - `riwayatTransaksi.add(...)` → pemanfaatan atribut `protected` dari kelas induk untuk mencatat entitas riwayat transaksi bonus awal.
 - `saldo` → pemanfaatan atribut `protected` dari kelas induk untuk menampilkan total saldo akhir di konsol.
- c. Alur Kerja Sintaks
- Saat objek RekeningVIP diinstansiasi, konstruktor langsung memanggil `super(...)` di baris pertama. Konstruktor induk Rekening menerima akumulasi saldo `saldoAwal + BONUS_VIP` sehingga nilai atribut `saldo` secara otomatis tersimpan dengan total nilai setelah bonus.
 - Sistem membuat ID transaksi unik berbasis timestamp (`System.currentTimeMillis()`) dengan prefiks "TRX-BONUS-".
 - Objek Transaksi baru untuk pencatatan bonus dimasukkan secara langsung ke dalam daftar `riwayatTransaksi` milik kelas induk.
 - Program mencetak pesan ucapan selamat serta total saldo gabungan (`saldoAwal + BONUS_VIP`) ke konsol.

2.2.5 Class 5 “Transaksi”

1. Kode Program

```
PrakPBO_B_2511531020 ▶ src ▶ pekan4 ▶ Transaksi ▶
1 package pekan4;
2
3 public class Transaksi {
4     //1. Mengubah semua atribut menjadi private
5     private String idTransaksi;
6     private String jenis;
7     private double nominal;
8
9     public Transaksi(String id, String jenis, double nominal) {
10         this.idTransaksi = id;
11         this.jenis = jenis;
12         this.nominal = nominal;
13     }
14     //2. Hanya menyediakan Getter (Read-only)
15     public String getIdTransaksi() {return idTransaksi;}
16     public String getJenis() {return jenis;}
17     public double getNominal() {return nominal;}
18
19     public void cetakDetail() {
20         System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
21     }
22 }
```

2. Analisis Langkah

a. Struktur Program

- Berfungsi sebagai model entitas data riwayat transaksi yang bersifat *immutable* (*read-only* setelah diinstansiasi) untuk mencatat setiap aktivitas keuangan pada rekening.

b. Elemen Sintaks Utama

- `private String idTransaksi, jenis dan private double nominal` → atribut dikunci dengan hak akses `private` agar nilai riwayat transaksi tidak dapat diubah dari luar kelas.
- `public Transaksi(...)` → konstruktor untuk menginisialisasi atribut ID transaksi, jenis transaksi, dan nominal saat objek dibuat.
- Method Getter (`getIdTransaksi()`, `getJenis()`, `getNominal()`) → menyediakan akses pembacaan nilai atribut tanpa memberikan method *setter*.
- `public void cetakDetail()` → metode untuk menampilkan rincian riwayat transaksi (ID, Jenis, dan Nominal) secara terformat ke konsol.

c. Alur Kerja Sintaks

- Saat objek `Transaksi` diinstansiasi melalui konstruktor, data ID transaksi, jenis, dan nominal secara permanen disimpan ke dalam atribut objek.

- Karena seluruh atribut ber-modifier `private` dan tidak disediakan method `setter`, nilai transaksi bersifat aman (*read-only*) dan tidak dapat dimodifikasi kembali.
- Pembacaan data dilakukan melalui method `getter` atau dengan memanggil `cetakDetail()` saat mencetak riwayat mutasi rekening.

2.2.6 Class 6 “Main”

1. Kode Program

```

1 package pekan4;
2
3 import java.util.ArrayList;
4
5
6 public class Main {
7     public static void main(String[] args) {
8         Scanner input = new Scanner(System.in);
9         ArrayList<Rekening> daftarRekening = new ArrayList<>();
10        Rekening akunAktif = null;
11        boolean isRunning = true;
12
13        System.out.println("=== SISTEM PERBANKAN MINI (INHERITANCE) ===");
14
15        while (isRunning) {
16            System.out.println("\nMenu Utama:");
17            System.out.println("1. Buka Rekening Baru");
18            System.out.println("2. Setor Tunai");
19            System.out.println("3. Tarik Tunai (Wajib PIN)");
20            System.out.println("4. Cek Informasi Rekening");
21            System.out.println("5. Ganti Akun");
22            System.out.println("6. Cetak Mutasi (Wajib PIN)");
23            System.out.println("7. Simulasi Akhir Bulan (Khusus Tabungan)");
24            System.out.println("8. Keluar");
25            System.out.print("Pilih menu: ");
26            int pilihan = input.nextInt();
27            input.nextLine(); // Clear buffer
28
29            switch (pilihan) {
30                case 1:
31                    System.out.print("Masukkan No Rekening: ");
32                    String no = input.nextLine();
33                    System.out.print("Masukkan Nama Pemilik: ");
34                    String nama = input.nextLine();
35                    System.out.print("Masukkan Saldo Awal: ");
36
37                    double saldo = input.nextDouble();
38                    input.nextLine();
39                    System.out.print("Masukkan PIN Baru (6 digit): ");
40                    String pinBaru = input.nextLine();
41
42                    // Dalam switch-case (case 1:) pada Main.java
43                    System.out.println("Pilih Produk Rekening:");
44                    System.out.println("1. Tabungan Umum (Bunga Bulanan)");
45                    System.out.println("2. Giro Bisnis (Fasilitas Overdraft)");
46                    System.out.println("3. Rekening VIP (Bonus Saldo Rp100.000)");
47                    System.out.print("Pilihan Produk (1/2/3): ");
48                    int jenisProduk = input.nextInt();
49                    input.nextLine();
50
51                    Rekening akunBaru = null;
52                    if (jenisProduk == 1) {
53                        System.out.print("Masukkan Suku Bunga Bulanan (%): ");
54                        double bunga = input.nextDouble();
55                        input.nextLine();
56                        akunBaru = new RekeningTabungan(no, nama, saldo, pinBaru, saldo);
57                    } else if (jenisProduk == 2) {
58                        System.out.print("Masukkan Batas Overdraft (Limit Pinjaman): ");
59                        double limit = input.nextDouble();
60                        input.nextLine();
61                        akunBaru = new RekeningGiro(no, nama, saldo, pinBaru, limit);
62                    } else if (jenisProduk == 3) {
63                        // Instansiasi Subclass RekeningVIP (Tugas Challenge No. 2)
64                        akunBaru = new RekeningVIP(no, nama, saldo, pinBaru);
65                    } else {
66                        System.out.println("Produk tidak valid! Menggunakan standar Rekening.");
67                        akunBaru = new Rekening(no, nama, saldo, pinBaru);
68                    }

```

```

69         // Upcasting: Simpan ke daftarRekening
70         daftarRekening.add(akunBaru);
71         akunAktif = akunBaru;
72         break;
73
74     case 2:
75         if (akunAktif == null) {
76             System.out.println("Error: Anda belum memilih/memiliki rekening!");
77         } else {
78             System.out.print("Masukkan nominal setor: ");
79             double setor = input.nextDouble();
80             akunAktif.setorTunai(setor);
81         }
82         break;
83
84     case 3:
85         if (akunAktif == null) {
86             System.out.println("Error: Anda belum memilih/memiliki rekening!");
87         } else {
88             System.out.print("Masukkan PIN Anda: ");
89             String pinInput = input.nextLine();
90             if (akunAktif.otentikasi(pinInput)) {
91                 System.out.print("Masukkan nominal tarik tunai: ");
92                 double tarik = input.nextDouble();
93                 akunAktif.tarikTunai(tarik);
94             } else {
95                 System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
96             }
97         }
98         break;
99
100     case 4:
101         if (akunAktif == null) {
102             System.out.println("Error: Anda belum memiliki rekening!");
103         } else {
104             akunAktif.cekInformasi();
105         }
106         break;
107
108     case 5:
109         if (daftarRekening.isEmpty()) {
110             System.out.println("Belum ada rekening yang terdaftar.");
111         } else {
112             System.out.print("Masukkan Nomor Rekening yang ingin diaktifkan: ");
113             String cariNo = input.nextLine();
114             boolean ditemukan = false;
115             for (Rekening r : daftarRekening) {
116                 if (r.getNomorRekening().equalsIgnoreCase(cariNo)) {
117                     akunAktif = r;
118                     ditemukan = true;
119                     System.out.println("Berhasil mengganti ke akun milik: " + r.getNamaPemilik());
120                     break;
121                 }
122             }
123             if (!ditemukan) {
124                 System.out.println("Error: Nomor rekening tidak ditemukan!");
125             }
126         }
127         break;
128
129     case 6:
130         if (akunAktif == null) {
131             System.out.println("Error: Anda belum memiliki rekening!");
132         } else {
133             System.out.print("Masukkan PIN Anda: ");
134             String pinInput = input.nextLine();
135             if (akunAktif.otentikasi(pinInput)) {
136                 akunAktif.cetakMutasi();
137             } else {
138                 System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
139             }
140         }
141         break;
142
143     case 7:
144         // Simulasi Akhir Bulan (Tugas 2)
145         if (akunAktif == null) {
146             System.out.println("Error: Anda belum memiliki rekening!");
147         } else {
148             // Pengecekan instanceof sebelum Downcasting
149             if (akunAktif instanceof RekeningTabungan) {
150                 // Downcasting eksplisit dari Rekening ke RekeningTabungan
151                 RekeningTabungan tabungan = (RekeningTabungan) akunAktif;
152                 tabungan.tambahBungaAkhirBulan();
153             } else {
154                 System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.");
155             }
156         }
157         break;
158
159     case 0:
160         isRunning = false;
161         System.out.println("Sistem ditutup. Terima kasih!");
162         break;
163
164     default:
165         System.out.println("Pilihan tidak valid!");
166     }
167     input.close();
168 }
169 }

```

2. Analisis Langkah

a. Struktur Program

- Berfungsi sebagai *driver class* (kelas utama) tempat eksekusi program dimulai. Kelas ini mengelola *user interface* berbasis konsol, pengolahan input pilihan menu, manajemen daftar rekening melalui *Upcasting*, serta penerapan *Downcasting* berbasis pengecekan tipe objek (*instanceof*).

b. Elemen Sintaks Utama

- `ArrayList<Rekening> daftarRekening = new ArrayList<>()` → struktur data koleksi untuk menyimpan berbagai jenis objek rekening (*polimorfisme/upcasting*).
- `Rekening akunAktif = null` → variabel penampung bertipe *superclass* `Rekening` untuk mencatat akun yang sedang dipilih/digunakan.
- `switch` (pilihan) → kontrol alur bercabang untuk mengeksekusi fitur perbankan sesuai menu yang dipilih pengguna.
- `akunAktif.otentikasi(pinInput)` → pemanggilan metode validasi PIN sebelum pengguna melakukan tindakan sensitif (Tarik Tunai dan Cetak Mutasi).
- `instanceof & (RekeningTabungan) akunAktif` → pengecekan tipe runtime objek dan proses *Downcasting* eksplisit dari `Rekening` ke `RekeningTabungan` sebelum memanggil fungsi spesifik `tambahBungaAkhirBulan()`.

c. Alur Kerja Sintaks

- Inisialisasi & Loop Utama: Program memulai perulangan `while (isRunning)` dan menampilkan menu interaktif perbankan hingga pengguna memilih angka 0 untuk keluar.
- Menu 1 (Buka Rekening Baru): Meminta input data dasar nasabah (nomor, nama, saldo, PIN) dan pilihan jenis produk. Sesuai pilihan:
 - Pilihan 1 instansiasi `RekeningTabungan`. (*Catatan: Pada argumen ke-5 kode di atas diisi saldo untuk parameter suku bunga*).
 - Pilihan 2 meminta batas limit lalu menginstansiasi `RekeningGiro`.

- Pilihan 3 menginstansiasi RekeningVIP (otomatis menambah bonus Rp100.000).
- Objek yang dibuat disimpan ke daftarRekening via *Upcasting* ke tipe Rekening dan dijadikan akunAktif.
- Menu 2 & 4 (Setor Tunai & Cek Informasi): Memastikan akunAktif tidak null, kemudian mengeksekusi metode setorTunai() atau cekInformasi().
- Menu 3 & 6 (Tarik Tunai & Cetak Mutasi): Meminta input PIN pengguna; jika otentikasi() mengembalikan nilai true, transaksi penarikan saldo atau pencetakan daftar mutasi dijalankan.
- Menu 5 (Ganti Akun): Melakukan pencarian (*linear search*) pada daftarRekening berdasarkan nomor rekening. Jika cocok, akunAktif diperbarui ke objek rekening tersebut.
- Menu 7 (Simulasi Akhir Bulan): Memeriksa apakah akunAktif merupakan instance dari RekeningTabungan menggunakan instanceof. Jika ya, dilakukan *downcasting* menjadi RekeningTabungan tabungan lalu menjalankan tambahBungaAkhirBulan(). Jika bukan RekeningTabungan, menampilkan pesan penolakan.

2.3 Latihan

2.3.1 TUGAS 1

1. Setelah user menginputkan Saldo Awal dan PIN, berikan opsi produk: "Pilih Produk: 1. Tabungan Umum | Giro Bisnis"

```

40
41 // Dalam switch-case (case 1:) pada Main.java
42 System.out.println("Pilih Produk Rekening:");
43 System.out.println("1. Tabungan Umum (Bunga Bulanan)");
44 System.out.println("2. Giro Bisnis (Fasilitas Overdraft)");
45 System.out.println("3. Rekening VIP (Bonus Saldo Rp100.000)");
46 System.out.print("Pilihan Produk (1/2/3): ");
47 int jenisProduk = input.nextInt();
48 input.nextLine();
49

```

Setelah menerima input data dasar nasabah (Nomor, Nama, Saldo, PIN), program memberikan pilihan jenis produk rekening yang ingin dibuat.

2. Jika user memilih Tabungan, minta input sukuBunga dalam persen, lalu instansiasi objek RekeningTabungan.

```

49         Rekening akunBaru = null;
50     if (jenisProduk == 1) {
51         System.out.print("Masukkan Suku Bunga Bulanan (%): ");
52         double bunga = input.nextDouble();
53         input.nextLine();
54         akunBaru = new RekeningTabungan(no, nama, saldo, pinBaru, saldo);
55     }

```

Program meminta persentase bunga bulanan, kemudian membuat objek baru dari kelas RekeningTabungan.

3. Jika user memilih Giro, minta input batasOverdraft (limit pinjaman), lalu instansiasi objek RekeningGiro.

```

56     } else if (jenisProduk == 2) {
57         System.out.print("Masukkan Batas Overdraft (Limit Pinjaman): ");
58         double limit = input.nextDouble();
59         input.nextLine();
60         akunBaru = new RekeningGiro(no, nama, saldo, pinBaru, limit);

```

Program meminta besaran limit pinjaman (*overdraft*), kemudian membuat objek baru dari kelas RekeningGiro.

4. Simpan objek tersebut ke dalam daftarRekening (Upcasting: Subclass otomatis dikenali sebagai tipe Superclass-nya).

```

69         // Upcasting: Simpan ke daftarRekening
70         daftarRekening.add(akunBaru);
71         akunAktif = akunBaru;
72         break;
73     }

```

Objek *subclass* (RekeningTabungan, RekeningGiro, atau RekeningVIP) disimpan ke dalam ArrayList<Rekening> daftarRekening. Proses penampungan objek kelas anak ke dalam referensi kelas induk (Rekening) ini dinamakan Upcasting.

2.3.2 TUGAS 2

1. Tambahkan menu baru: "7. Simulasi Akhir Bulan (Khusus Tabungan)" di CLI.

```

141         break;
142     case 7:
143         // Simulasi Akhir Bulan (Tugas 2)
144         if (akunAktif == null) {
145             System.out.println("Error: Anda belum memiliki rekening!");
146         } else {
147             // Pengecekan instanceof sebelum Downcasting
148             if (akunAktif instanceof RekeningTabungan) {
149                 // Downcasting eksplisit dari Rekening ke RekeningTabungan
150                 RekeningTabungan tabungan = (RekeningTabungan) akunAktif;
151                 tabungan.tambahBungaAkhirBulan();
152             } else {
153                 System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.");
154             }
155         }
156         break;
157     }

```

Pilihan menu nomor 7 ditambahkan ke dalam tampilan Menu Utama di kelas Main.java.

2. Di dalam case 7:, periksa apakah akunAktif saat ini adalah objek Tabungan menggunakan operator instanceof.

```

147         // Pengecekan instanceof sebelum Downcasting
148         if (akunAktif instanceof RekeningTabungan) {
149             // Downcasting eksplisit dari Rekening ke Rek

```

program memeriksa apakah objek yang sedang aktif (akunAktif) merupakan instansiasi asli dari kelas RekeningTabungan.

3. Jika akun tersebut benar RekeningTabungan, lakukan Downcasting ke tipe Tabungan, lalu panggil method tambahBungaAkhirBulan().

```

149         // Downcasting eksplisit dari Rekening ke RekeningTabungan
150         RekeningTabungan tabungan = (RekeningTabungan) akunAktif;
151         tabungan.tambahBungaAkhirBulan();

```

Karena objek akunAktif bertipe umum Rekening, program melakukan Downcasting secara eksplisit menjadi (RekeningTabungan) agar dapat mengeksekusi method khusus tambahBungaAkhirBulan() milik RekeningTabungan.

4. Jika bukan, tolak dengan pesan: "Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan."

```

2         } else {
3             System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.");
4         }

```

Jika akunAktif adalah tipe rekening lain (seperti RekeningGiro atau RekeningVIP), program mengeksekusi blok else dan menampilkan pesan bahwa fitur bunga tidak dapat digunakan.

2.4 OUTPUT

1. Skenario A: Buka Rekening Tabungan & Jalankan Simulasi Akhir Bulan (Berhasil)

Main (4) [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-1412\jre\bin\javaw.exe (Oct 4
 === SISTEM PERBANKAN MINI (INHERITANCE) ===

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai (Wajib PIN)
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Wajib PIN)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 101
Masukkan Nama Pemilik: Dinda Amelia
Masukkan Saldo Awal: 1000000
Masukkan PIN Baru (6 digit): 123456
Pilih Produk Rekening:
1. Tabungan Umum (Bunga Bulanan)
2. Giro Bisnis (Fasilitas Overdraft)
3. Rekening VIP (Bonus Saldo Rp100.000)
Pilihan Produk (1/2/3): 1
Masukkan Suku Bunga Bulanan (%): 2
Rekening atas nama Dinda Amelia berhasil dibuat.
..

```

2. Buka Rekening Giro & Jalankan Simulasi Akhir Bulan (Gagal / Ditolak)

```
Main (4) [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-1412\jre\bin\javaw.exe (Oct 4, 2025)
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai (Wajib PIN)
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Wajib PIN)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 102
Masukkan Nama Pemilik: Dika
Masukkan Saldo Awal: 5000000
Masukkan PIN Baru (6 digit): 654321
Pilih Produk Rekening:
1. Tabungan Umum (Bunga Bulanan)
2. Giro Bisnis (Fasilitas Overdraft)
3. Rekening VIP (Bonus Saldo Rp100.000)
Pilihan Produk (1/2/3): 2
Masukkan Batas Overdraft (Limit Pinjaman): 2000000
Rekening atas nama Dika berhasil dibuat.

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai (Wajib PIN)
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Wajib PIN)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 7
Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.
```

BAB III

KESIMPULAN

Penerapan konsep *Inheritance*, *Encapsulation*, dan *Polymorphism* pada program perbankan ini berhasil membangun struktur kode yang aman, rapi, dan modular. Penggunaan modifier *protected* pada kelas induk Rekening memudahkan kelas turunan (RekeningTabungan, RekeningGiro, dan RekeningVIP) dalam mengelola data saldo secara langsung tanpa mengabaikan aspek keamanan data. Selain itu, mekanisme *Upcasting* memungkinkan seluruh jenis produk rekening disimpan ke dalam satu koleksi `ArrayList<Rekening>` secara praktis, sehingga mempermudah pengelolaan multi-produk dalam satu sistem terpadu.

Di sisi lain, penerapan *Downcasting* yang dikombinasikan dengan pengecekan operator `instanceof` terbukti efektif dalam menjamin *type safety* saat menjalankan fungsi spesifik, seperti kalkulasi bunga bulanan pada RekeningTabungan. Poin penting dari pengerjaan tugas ini adalah ketelitian dalam pengiriman argumen parameter konstruktor. Perbaikan passing parameter pada variabel bunga serta pembentukan format desimal terbukti krusial agar proses perhitungan bunga dan penampilannya pada output program berjalan dengan akurat dan logis.

DAFTAR PUSTAKA

Nurfiah. (2026). Modul Praktikum Pemrograman Berorientasi Objek (Modul 4: Ekspansi Produk Keuangan - Inheritance). Laboratorium Komputer, Departemen Informatika, Universitas Andalas.